PET ADOPTION PLATFORM

Software Requirements Specification (SRS)
and UML Design Document

Project Type	Web-based application for discovering pets and managing adoption requests
Document Version	1.0 — Academic Project
Prepared For	Software Engineering / GitHub Project Repository
Date	October 2026

Purpose: define the system scope, requirements, workflows, data model, and UML views needed to implement a transparent and user-friendly pet adoption process.

Document Contents

Section	Topic
1	Introduction
2	Overall Description
3	Functional Requirements
4	Non-Functional Requirements
5	Data Requirements and Business Rules
6	Use Case Model
7	Class Diagram
8	Sequence Diagram
9	Activity Diagram
10	Adoption Request State Diagram
11	Context DFD
12	Deployment Diagram
13	Acceptance Criteria
14	Assumptions, Limitations and Future Enhancements

Document note: this is a project-ready baseline. Update the implementation stack, team details, and screenshots to match your actual repository before submission.

1. Introduction

1.1 Purpose. This Software Requirements Specification describes the expected behaviour and quality requirements of the Pet Adoption Platform. It is intended for students, developers, testers, project reviewers, and future maintainers.

1.2 Problem Statement. Pet adoption information is often scattered across social media, phone calls, and local shelter records. Interested adopters may struggle to find reliable pet details, while shelters may find it difficult to track applications consistently. The platform centralises listings and adoption-request status.

1.3 Project Objectives

- Allow adopters to browse and search available pets.
- Provide consistent pet profiles with photos, age, species, health notes, and shelter contact details.
- Let eligible adopters submit and track adoption requests.
- Allow shelter staff to manage pet listings and review applications.
- Provide administrator controls for users, listings, and basic moderation.

1.4 Scope. The system covers account access, pet catalogue, search/filter, adoption application, application review, status notifications, and administrative moderation. Physical handover, veterinary treatment, payment processing, and legal ownership transfer remain outside the system scope.

1.5 Definitions

Term	Meaning
Adopter	A registered user interested in adopting a pet.
Shelter Staff	An authorised representative who maintains pet listings and reviews requests.
Adoption Request	A submitted application to adopt a specific pet.
Administrator	A privileged user responsible for platform-level moderation.
Pet Status	Availability state such as Available, Pending, Adopted, or Inactive.

2. Overall Description

2.1 Product Perspective. The platform is a standalone web application with a browser-based client, application/API layer, and persistent database. Pet images may be stored in file or object storage, while the database stores image references.

2.2 User Classes

User class	Responsibilities	Access level
Guest	Browse available pets and view public profiles.	Read-only public access
Adopter	Create account, submit request, view own request history.	Own profile and requests
Shelter Staff	Create/edit pet listings, review adoption requests, update decisions.	Assigned shelter records
Administrator	Moderate listings, manage accounts, view platform-level reports.	Administrative functions

2.3 Operating Environment. Modern mobile or desktop browser; internet connection; server environment supporting an application API and relational database. The design is technology-neutral and can be implemented using Java, Python, PHP, JavaScript, or another suitable stack.

2.4 Constraints. Users must not access another adopter's private application data. Only authorised shelter staff may change review decisions. Pet availability must be checked before an application is accepted. Personal data should be collected only when needed for adoption processing.

2.5 Assumptions and Dependencies. Shelters provide accurate pet details and have authorised staff accounts. Email/SMS notifications depend on a configured notification provider. Image storage and backups are available in the deployment environment.

3. Functional Requirements

ID	Requirement
FR-01	The system shall allow an adopter to register using a name, email address, and password.
FR-02	The system shall authenticate registered users and provide role-based access.
FR-03	The system shall display available pets with a profile page and photos where available.
FR-04	The system shall allow filtering by species, age range, location, and availability.
FR-05	The system shall allow an authenticated adopter to submit an application for an available pet.
FR-06	The system shall validate required application fields and prevent duplicate active requests for the same pet by the same adopter.
FR-07	The system shall show each adopter the status of their own requests.
FR-08	Shelter staff shall be able to add, edit, deactivate, and update pet availability.
FR-09	Shelter staff shall be able to review an application and mark it Approved or Rejected with an optional note.
FR-10	The system shall notify the adopter when the request status changes, where notifications are configured.
FR-11	Administrators shall be able to suspend accounts and moderate reported or inappropriate listings.
FR-12	The system shall record timestamps and actor IDs for important status changes.

4. Non-Functional Requirements

ID	Quality attribute	Requirement
NFR-01	Usability	Common actions should use clear labels, validation messages, and consistent navigation.
NFR-02	Performance	For a normal connection and typical project dataset, common catalogue searches should return within 3 seconds.
NFR-03	Security	Passwords must be securely hashed; authenticated endpoints must enforce role and ownership checks.
NFR-04	Privacy	Only authorised users may view private application details; secrets must not be committed to GitHub.
NFR-05	Reliability	The system should validate data before saving and show a clear error if an operation fails.
NFR-06	Availability	The deployed service should target 99% monthly availability, excluding planned maintenance, if hosting permits.
NFR-07	Maintainability	Business logic, user interface, and data access should be separated into maintainable modules.
NFR-08	Compatibility	The interface should work on current major browsers and adapt to mobile and desktop screens.
NFR-09	Accessibility	Form fields should have labels, keyboard focus should be visible, and important information should not rely on colour alone.
NFR-10	Auditability	Application status changes should record the previous state, new state, timestamp, and responsible staff account.

5. Data Requirements and Business Rules

5.1 Main entities. User, Shelter, Pet, AdoptionRequest, PetImage, and Notification. An implementation may add address, vaccination, audit-log, and report entities if required.

5.2 Business rules

- A pet can receive requests only while its status is Available.
- An adopter can see only their own private application information.
- A shelter staff member can manage only shelters assigned to that account, unless an administrator grants broader access.
- An approved adoption should trigger a pet availability update or a staff confirmation step to avoid conflicting approvals.
- Rejected or withdrawn requests remain in history for traceability; they are not silently deleted.
- Every status transition must be validated against the permitted state flow.

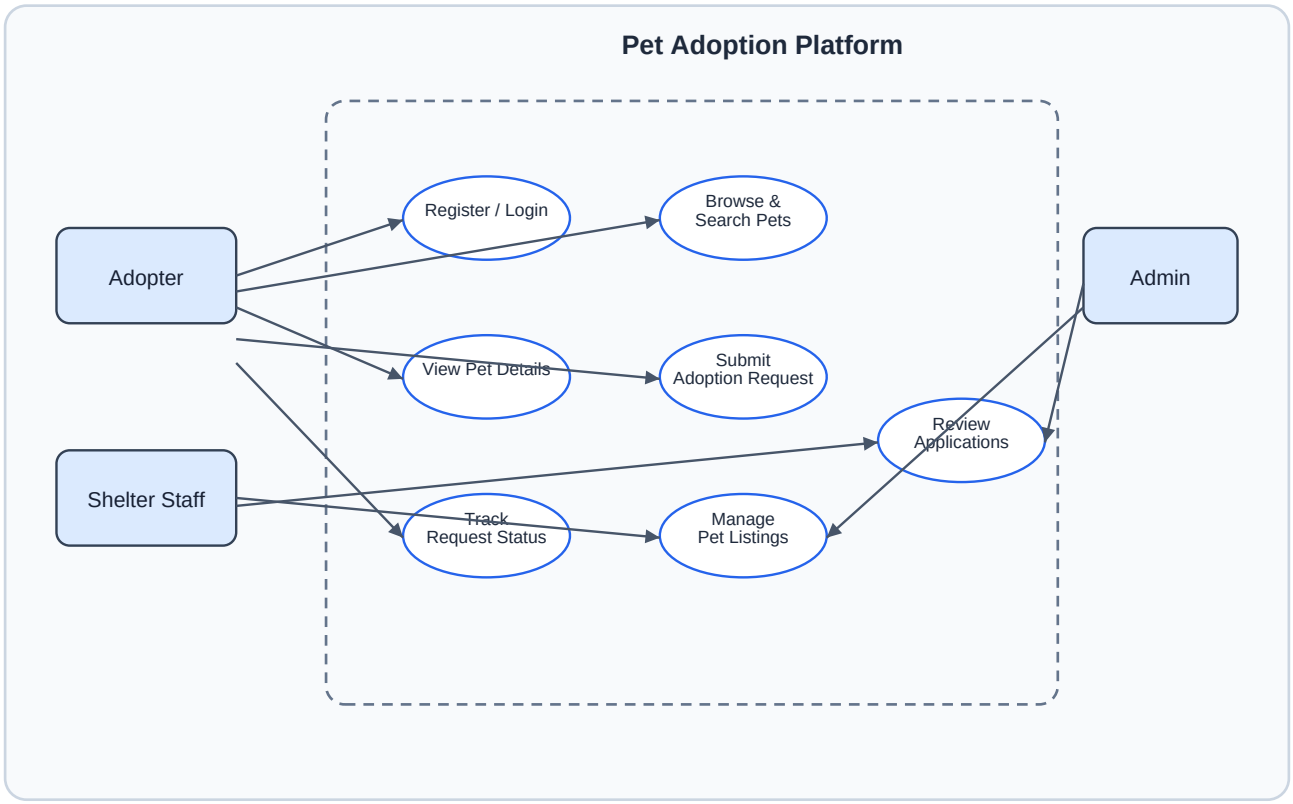
5.3 Suggested data dictionary

Entity	Key fields
User	user_id (PK), full_name, email (unique), password_hash, role, created_at, active
Shelter	shelter_id (PK), name, location, phone, email, staff_user_id (FK)
Pet	pet_id (PK), shelter_id (FK), name, species, breed, age, description, health_notes, status, created_at
AdoptionRequest	request_id (PK), pet_id (FK), adopter_id (FK), submitted_at, status, applicant_note, reviewer_note, reviewed_by (FK), reviewed_at

Entity	Key fields
PetImage	image_id (PK), pet_id (FK), image_url, alt_text
Notification	notification_id (PK), user_id (FK), message, created_at, read_at

6. UML Use Case Diagram

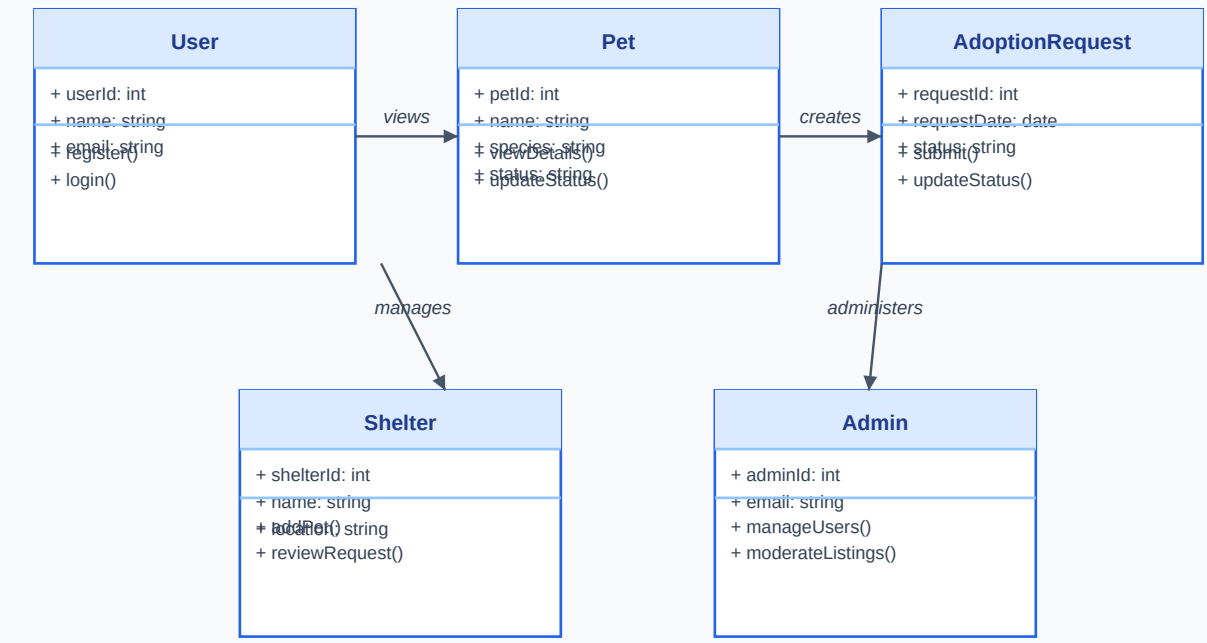
The use case model shows the main services offered by the platform and the roles that interact with them. Guests can browse public listings; adopters can submit and track requests; shelter staff manage pets and review applications; administrators perform platform moderation.



Primary use cases: Register/Login, Browse & Search Pets, View Pet Details, Submit Adoption Request, Track Request Status, Manage Pet Listings, Review Applications, and Moderate Users/Listings.

7. UML Class Diagram

The class diagram identifies the main domain objects, their key attributes, and representative operations. The model can be mapped to database tables and application classes.

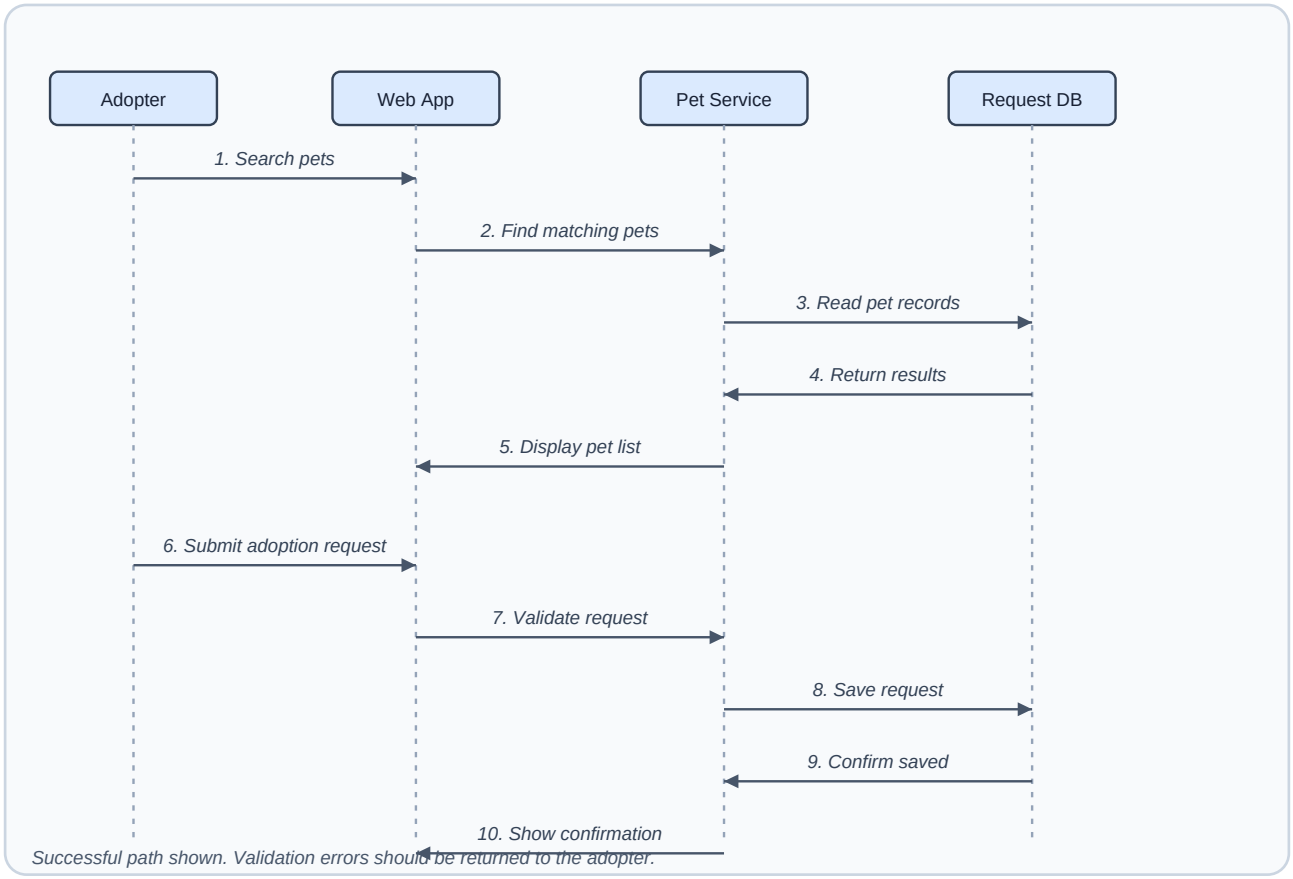


Associations shown conceptually; request is submitted by a user for a pet.

Relationship notes: a User may submit many AdoptionRequests; each request refers to one Pet. A Shelter maintains multiple Pets. An administrator performs moderation across the platform. The diagram is a conceptual model and can be refined with explicit multiplicity labels during implementation.

8. UML Sequence Diagram — Search and Apply

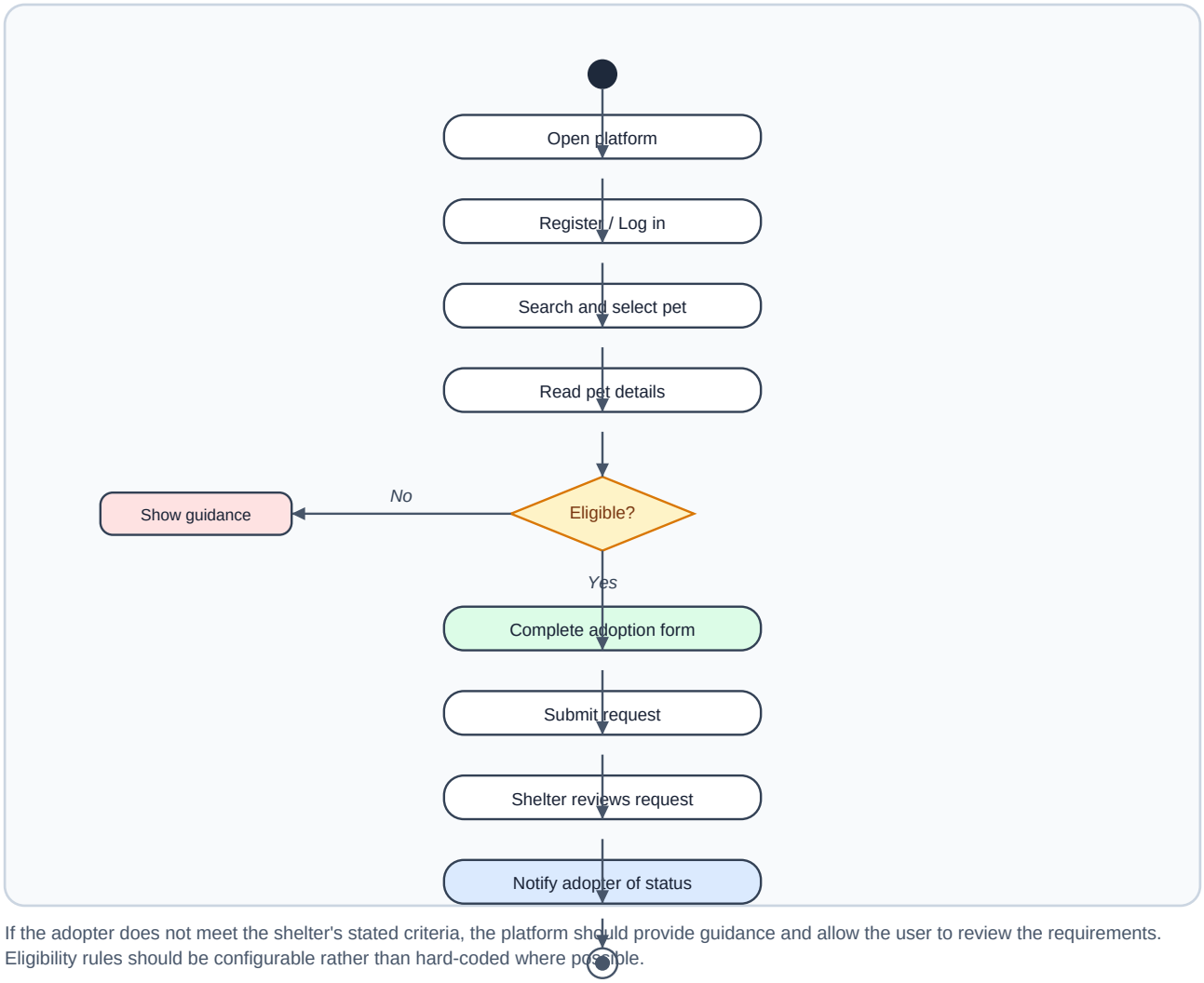
This sequence shows the successful path from searching the pet catalogue to saving an adoption request. In the implemented application, validation failures should return a useful message without creating a partial request.



Preconditions: the adopter is signed in and the pet is available. **Postcondition:** a request record is stored and the adopter receives a confirmation message or sees a confirmation screen.

9. UML Activity Diagram — Adoption Workflow

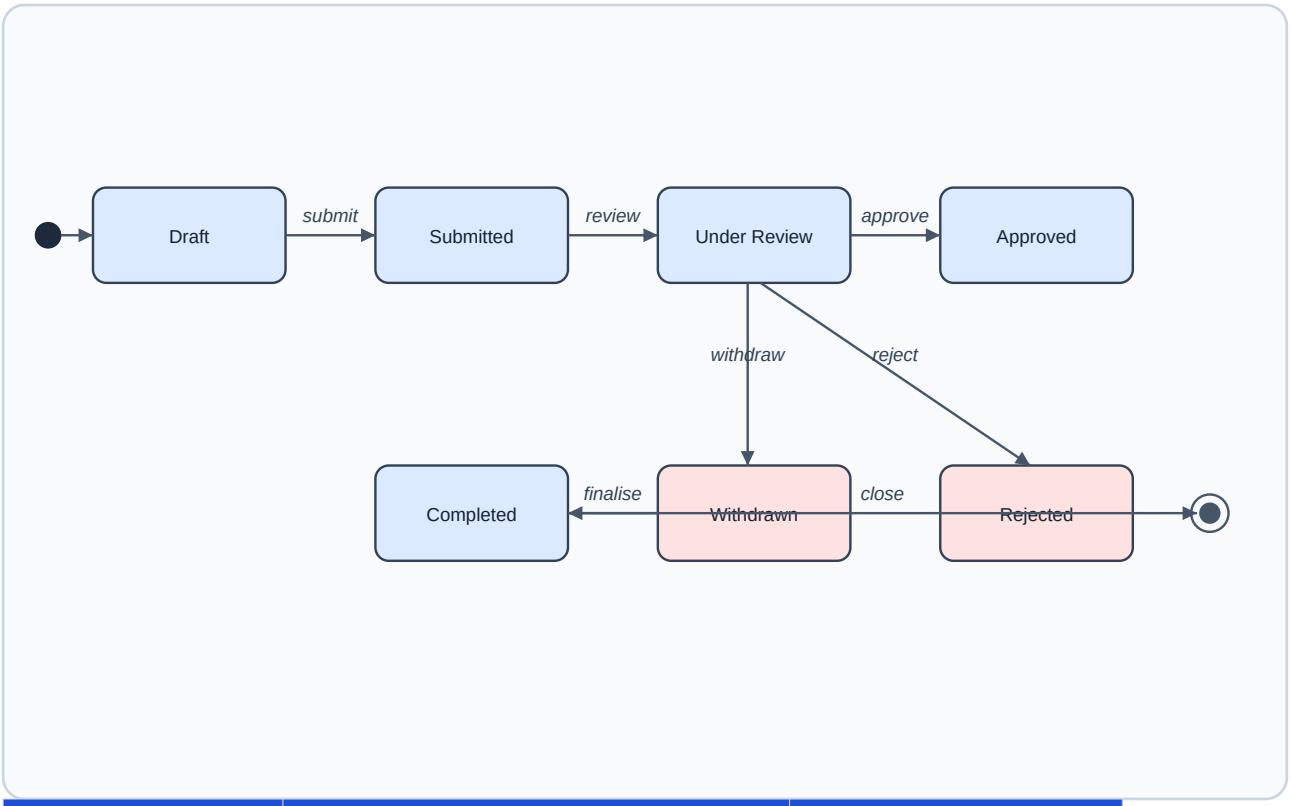
The activity diagram captures the end-to-end workflow, including the eligibility decision, application submission, shelter review, and status notification.



If the adopter does not meet the shelter's stated criteria, the platform should provide guidance and allow the user to review the requirements. Eligibility rules should be configurable rather than hard-coded where possible.

10. UML State Diagram — Adoption Request

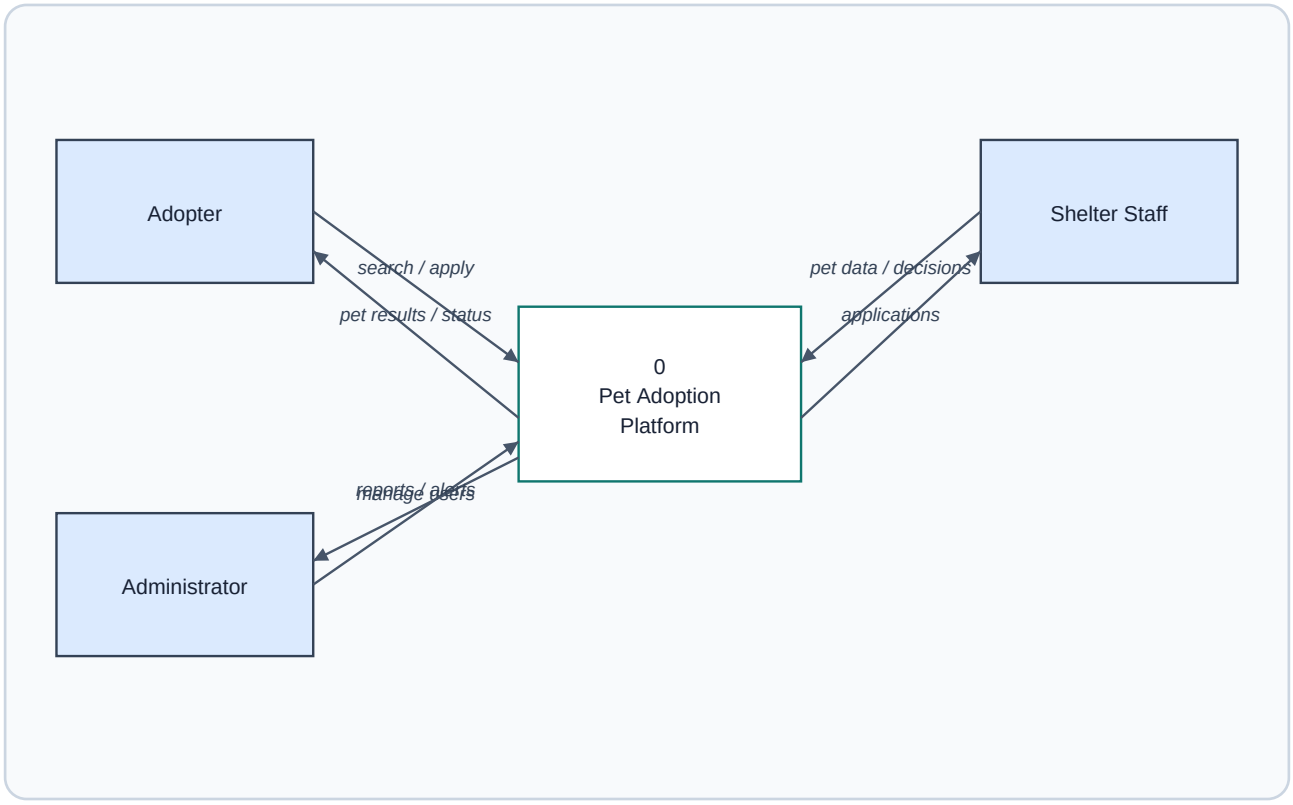
An adoption request moves through controlled states. The application must reject invalid transitions and retain a history of decisions.



Current state	Event / condition	Next state
Draft	Adopter submits valid form	Submitted
Submitted	Shelter begins assessment	Under Review
Under Review	Shelter accepts request	Approved
Under Review	Shelter declines request	Rejected
Submitted / Under Review	Adopter withdraws before final decision	Withdrawn
Approved	Handover is confirmed by authorised staff	Completed

11. Context Data Flow Diagram (Level 0)

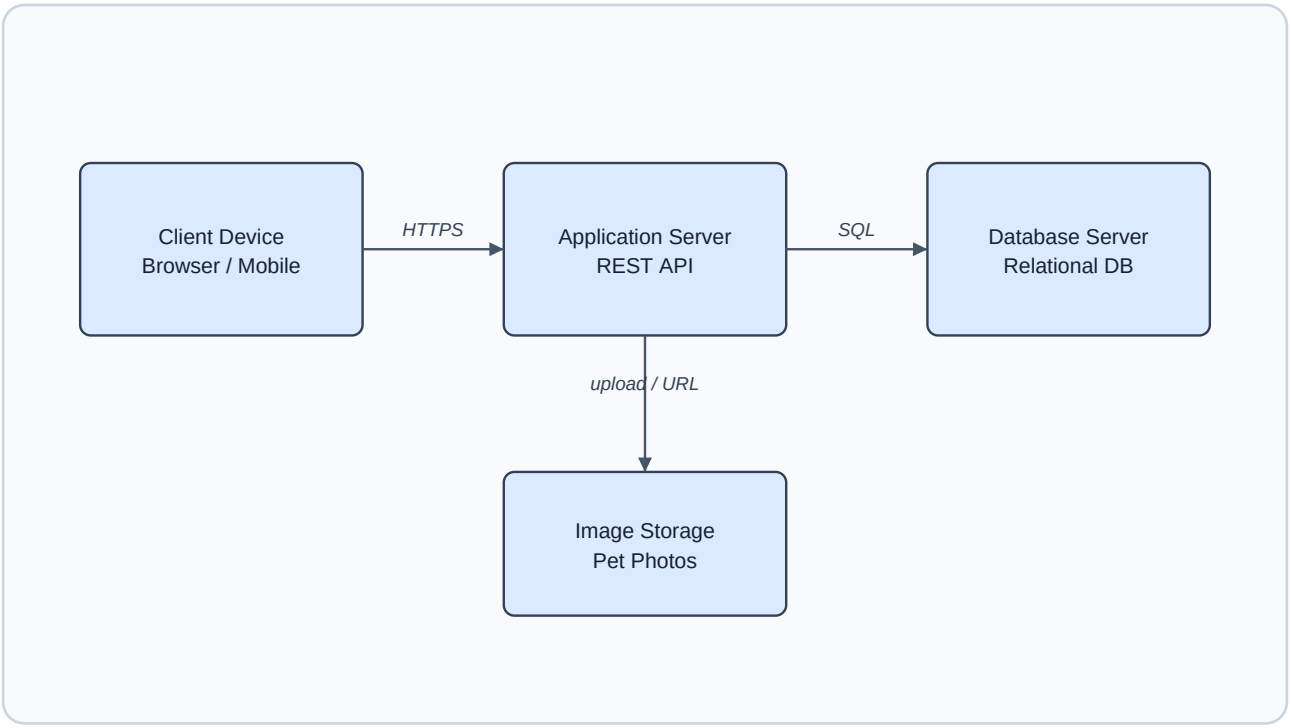
The context diagram treats the Pet Adoption Platform as one process and shows the main external actors and information exchanged. A detailed Level 1 DFD can be added later to show internal processes and data stores.



Typical data flows: pet search criteria, application details, pet listings, review decisions, status updates, account administration, and basic reports.

12. UML Deployment Diagram

The deployment diagram proposes a simple three-tier deployment. This is a logical architecture and does not require a specific cloud provider or programming language.



Deployment considerations: use HTTPS; store credentials in environment variables or a secret manager; restrict database access to the application server; back up the database; validate uploaded image type and size; and avoid committing real user data or API keys to source control.

13. Acceptance Criteria

Test ID	Acceptance check	Expected result
AT-01	Guest opens pet catalogue.	Available pet listings are visible without exposing private application data.
AT-02	Adopter searches by species and location.	Results match the selected filters or a clear no-results message appears.
AT-03	Unauthenticated visitor attempts to submit an application.	System requests login and does not create a request.
AT-04	Adopter submits a complete valid application for an available pet.	Request is saved once and a confirmation is shown.
AT-05	Adopter attempts to view another user's request.	Access is denied.
AT-06	Shelter staff approves or rejects a request.	Status, reviewer, and timestamp are saved; adopter can see the updated status.
AT-07	Staff tries to edit an unrelated shelter's listing.	System denies the operation unless the staff account has permission.
AT-08	Required field is missing.	Validation message is shown and invalid data is not saved.

14. Assumptions, Limitations and Future Enhancements

Assumptions

- Shelters are responsible for the truthfulness and freshness of listing information.
- The project has at least one administrator and one shelter staff account for demonstration.
- Email/SMS delivery may be simulated in a college demo if no notification provider is configured.

Current limitations

- The document describes requirements and design, not a deployed production service.
- Identity verification, legal agreements, payments, and in-person adoption handover are outside the defined scope.
- Performance targets should be verified against the actual hosting environment and test dataset.

Future enhancements

- Appointment scheduling for shelter visits.
- Vaccination and medical record attachments.
- Saved pets, comparison, and recommendation features.
- Multilingual interface and accessible content improvements.
- Email/SMS notifications, analytics, and verified shelter badges.
- Detailed Level 1 DFD, component diagram, and automated test traceability matrix.

Appendix A — Suggested GitHub Repository Structure

```
pet-adoption-platform/
■■■ README.md
■■■ docs/
■ ■■■ Pet_Adoption_Platform_SRS_UML.pdf
■■■ diagrams/ (optional editable source files)
■■■ src/ (application source code)
■■■ tests/ (test cases)
■■■ .gitignore
```

GitHub publishing checklist: add a README with project overview, features, technology stack, setup steps, screenshots, and team credits. Do not upload passwords, API keys, private adopter data, or real shelter contact data without permission. If you create editable diagrams separately, keep their source files in the repository too.

End of document